

PUCaronas

Plano de Teste

Versão 1.2

Histórico da Revisão

Data	Versão	Descrição	Autor
18/09/2026	1.0	Introdução do documento	Gustavo Pizato
19/09/2026	1.1	Organização e desenvolvimento dos casos de teste iniciais	Adriano, Gustavo, Matheus, Bernardo, João e Murilo
24/09/2026	1.2	Organização e desenvolvimento dos casos de teste finais	Adriano, Gustavo, Matheus, Bernardo, João e Murilo

Plano de Teste

1. Introdução

1.1. Objetivo:

O plano de testes tem como objetivo garantir a qualidade do site PUCaronas por meio de testes de funcionalidade, desempenho, acessibilidade, segurança, portabilidade, testes unitários e testes de API visando identificar possíveis falhas e avaliar a qualidade do sistema.

1.2. Escopo:

Serão testados as funcionalidades de carona, corrida, avaliação, os endpoints da pasta api/ em usuário, as triggers e procedures definidas em db.sql, o controle de acesso e o comportamento do sistema sob carga e seu desempenho. O objetivo é validar as regras de negócio, a integridade dos dados, as permissões de acesso e possíveis problemas no fluxo de aplicação para carona.

1.3. Abordagem:

Serão utilizados dois tipos principais de teste: Caixa Branca, aplicada aos testes de funcionalidade, desempenho e unitários, analisando como os registros são processados e carregados em cada página e respostas de API, além de testar diretamente no MySQL as triggers e procedures que estão em db.sql, comparando as tabelas antes e após sua execução e a Caixa Preta, aplicada também aos testes funcionais, com foco na validação do comportamento do sistema e na sua aderência aos requisitos definidos no PBI, sem considerar sua implementação interna.

1.4. Critérios:

O PUCaronas será considerado reprovado caso tenha a ocorrência de reprovação em casos classificados como alto, sendo suficiente a presença de uma única reprovação desta categoria. Casos de prioridade média ou inferior não impedem a conclusão dos testes. Nos testes de desempenho, o caso será considerado aprovado quando o tempo de carregamento de qualquer registro não ultrapassar 2 segundos. Para os demais testes, o caso será considerado aprovado quando o comportamento observado estiver de acordo com o resultado esperado e com os requisitos definidos no PBI; caso contrário, será considerado reprovado e o defeito deverá ser registrado para análise e correção.

2. Estratégia

2.1. Documentos de referência:

Foram utilizados como base para a elaboração deste plano de testes a documentação de especificação do projeto, incluindo a especificação de requisitos, o Canvas PBB, as User Stories e os diagramas de modelagem de atividades que estão localizados na pasta do google drive junto às evidências.

2.2. Ambiente de teste:

Elemento de Software	Versão	Tipo e Outras Observações
Visual Studio Code	1.138	Ambiente de desenvolvimento e edição do código
XAMPP Control Panel	3.3.0	Ambiente local para execução do servidor e banco de dados
Google Chrome	153.0.8010.48	Navegador utilizado para a execução dos testes
GitHub	https://github.com/AdrianoProenca06/testes-pucaronas	Plataforma de versionamento e gerenciamento do código-fonte
MySQL	8.0	Sistema de gerenciamento do banco de dados utilizado nos testes
Google Lighthouse	13.4.1	Sistema de testes automatizados e de código aberto voltado para a melhoria da qualidade de páginas web.
Google Drive	 testes_ver_val	Serviço utilizado para armazenamento e organização das evidências dos testes, como vídeos e imagens.
Postman	12.29.0	Serviço utilizado para testes unitários de API
Ferramenta de Captura do Windows	11.2508.29.0	Ferramenta utilizada para captura de telas e registro das evidências dos testes.
OBS Studio	32.2.2	Ferramenta utilizada para captura de tela

2.3. Processo adotado:

O processo de teste será composto por quatro etapas principais, que organizam a execução deste Plano de Teste:

Etapa	Descrição
Planejamento	Definição do escopo, tipos de teste, recursos, responsáveis e critérios de aprovação.
Preparação	Configuração do ambiente, ferramentas, dados e casos de teste necessários.
Execução	Realização dos testes conforme os casos e técnicas definidos no plano.
Documentação de evidências	Registro dos resultados, evidências, falhas encontradas e respectivas informações para acompanhamento e correção.

3. Planejamento

3.1. Componentes:

Esta seção apresenta a organização dos testes que serão realizados no PUCaronas, relacionando cada funcionalidade aos respectivos tipos de teste, técnicas utilizadas e responsáveis. O primeiro número do ID de cada caso indica o responsável (1 Gustavo, 2 Matheus, 3 Adriano, 4 Murilo, 5 João, 6 Bernardo). O objetivo é definir de forma eficiente o que será testado, como será testado e por quem, abrangendo testes funcionais, segurança, acessibilidade, portabilidade, desempenho, api e unitários.

Funcionalidade	Tipo de teste	Técnica	Responsável
Login de usuário	Funcionalidade	caixa-branca, caixa-preta, manual	01 - Gustavo
Cadastro de usuário	Funcionalidade	caixa-branca, manual	01 - Gustavo
Login de usuário	API - Unitário	caixa-branca, manual	01 - Gustavo

Cadastro de usuário	API - Unitário	caixa-branca,manual	01 - Gustavo
Criptografia de senhas	Segurança	caixa-branca,manual	01 - Gustavo
Veículo (adicionar, atualizar, excluir)	Funcionalidade	caixa-preta, manual	02 - Matheus
Veículo (listar, editar)	Desempenho	caixa-preta, automático	02 - Matheus
Procedure aprovar_usuario	Unitário	caixa-branca, manual	02 - Matheus
Veículo (Meus Carros)	Acessibilidade	caixa-preta, automático	02 - Matheus
Carona (consultar, adicionar, atualizar, excluir)	Funcionalidade	caixa-preta, manual	03 - Adriano
Carona (listar)	Desempenho	caixa-preta, automático	03 - Adriano
Carona (consultar)	Acessibilidade	caixa-preta, automático	03 - Adriano
Procedure deletar_carona	Unitário	caixa-branca, manual	03 - Adriano
Candidatura (adicionar, consultar, atualizar)	Funcionalidade	caixa-branca, caixa-preta, automático	04 - Murilo
Candidatura (listar)	Desempenho	caixa-branca, automático	04 - Murilo
Caronas (filtrar)	Funcionalidade	caixa-branca, automático	04 - Murilo
Trigger aplicacao_corrida	Unitário	caixa-branca, automático	04 - Murilo

Avaliação (adicionar, consultar)	Funcionalidade	caixa-branca, caixa-preta, automático	05 - João
Corrida (listar)	Stress	caixa-branca, automático	05 - João
Avaliação (adicionar)	Funcionalidade	caixa-preta, automático	05 - João
Avaliação	API - Unitário	caixa-preta, manual	05 - João
Trigger usuario_status	Unitário	caixa-branca, automático	05 - João
Corrida (consultar, atualizar, excluir)	Funcionalidade	caixa-branca, caixa-preta, automático	06 - Bernardo
Procedure deletar_usuari o (cascata em motorista e passageiro)	Unitário	caixa-branca, automático	06 - Bernardo

3.2. Cronograma:

Milestone	Data de Início	Data de Término	Responsável
Desenvolvimento de 70% do projeto	18/09/2026	23/09/2026	Adriano, Bernardo, Gustavo, Murilo, João e Matheus
Desenvolvimento dos 30% restantes do projeto.	25/09/2026	31/09/2026	Adriano, Bernardo, Gustavo, João, Murilo e Matheus

4. Casos de teste:

Regras para execução de caso de teste:

Guardar execução de casos de funcionalidade como vídeo no drive

O link das evidências está aqui: [📁 testes_ver_val](#)

4.1 Casos de funcionalidade:

ID	1.1
Cenário	Login de usuário
Caso de teste	Realizar o login de usuário em espera
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1. Acesse a página Registrar-se 2. Crie uma conta 3. Faça Login na mesma conta
Resultado esperado	O sistema deve exibir uma mensagem “usuário em espera” e levar para a tela home
Resultado obtido	O sistema levou o usuário para a página home.
Status	Aprovada
Observação	Evidências:  1.mp4

ID	1.2
Cenário	Login de usuário
Caso de teste	Realizar o login de usuário em banido
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Faça login como usuário não banido em espera. 1.1. Faça login como usuário Admin 1.2 Saia da conta do usuário em espera. 1.3 Faça um banimento do usuário em espera 1.4 Tente logar no usuário em espera
Resultado esperado	O sistema deve exibir “Usuário banido ou restringido”
Resultado obtido	O sistema exibiu “Você foi banido do site”, está incoerente com a especificação
Status	Reprovado
Observação	Evidências:  1.2.mp4

ID	1.3
Cenário	Cadastro de usuário
Caso de teste	Realizar o cadastro de usuário com um email maior de 100 caracteres
Prioridade	Média
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Entre na página de Cadastro 1.1 Preencha o e-mail de forma que ele seja superior 100 caracteres 1.2 Preencha o resto dos campos como passageiro. 1.3 Clique em enviar.
Resultado esperado	O sistema deve exibir “Dados inválidos!!!”
Resultado obtido	Não houve resposta do sistema ao clicar em enviar.
Status	Reprovado
Observação	Evidências: 📺 1.3.mp4 Foi impresso no console: “Resposta: Object { error: "Data too long for column 'email' at row 1" }”

ID	1.4
Cenário	Cadastro de usuário
Caso de teste	Realizar o cadastro com e-mail não institucional
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Entre na página de Cadastro 1.1 Preencha o e-mail usando um domínio @gmail.com 1.2 Preencha o resto dos campos como passageiro. 1.3 Clique em enviar.
Resultado esperado	O sistema deve exibir “E-mail inválido! Apenas e-mails do domínio @ pucpr.edu.br são permitidos”
Resultado obtido	O sistema exibiu “E-mail inválido! Apenas e-mails do domínio @ pucpr.edu.br são permitidos” , está coerente com o código
Status	Aprovado
Observação	Evidências: 📺 1.4.mp4

ID	2.1
Cenário	Gerir Veículo
Caso de teste	Adicionar um veículo no sistema
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	ADICIONAR 1.0. Acesse a área de meus veículos 1.1. Selecionar a opção Adicionar novo Carro 1.2. Preencha as informações de (modelo, placa e numero de assentos) 1.3 Clique em Cadastrar
Resultado esperado	O sistema deve permitir que o usuário logado como motorista consiga adicionar um veículo
Resultado obtido	O sistema permitiu o usuário motorista adicione um veículo
Status	Aprovado
Observação	Evidências: 📹 Teste ID 2.mov(0:00 - 0:31) Para adicionar um veículo o usuário deve estar logado como motorista

ID	2.2
Cenário	Gerir Veículo
Caso de teste	Excluir um veículo no sistema
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	EXCLUIR 1.0 Acesse a área de meus veículos 1.1 Selecione a opção DELETAR no veículo desejado 1.2 Confirme a exclusão clicando em EXCLUIR novamente
Resultado esperado	O sistema deve permitir que o usuário logado como motorista consiga deletar os seus veículos cadastrados
Resultado obtido	O sistema permitiu o usuário motorista exclua um veículo
Status	Aprovado
Observação	Evidências: 📹 Teste ID 2.mov(0:43 - 0:51)

	Para excluir um veículo o usuário deve estar logado como motorista
--	--

ID	2.3
Cenário	Gerir Veículo
Caso de teste	Atualizar dados de um veículo no sistema
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	ATUALIZAR 2.0 Acesse a área de meus veículos 2.1 Selecione a opção EDITAR no veículo desejado 2.2 Edite o veículo com as informações desejadas 2.3 Clique em Salvar Alterações
Resultado esperado	O sistema deve permitir que o usuário logado como motorista consiga atualizar os dados de seus veículos cadastrados
Resultado obtido	O sistema permitiu que o usuário logado como motorista atualize os dados de seu veículo desejado.
Status	Aprovado
Observação	Evidências: 📹 Teste ID 2.mov(0:31 - 0:43) Para editar um veículo o usuário deve estar logado como motorista

ID	2.4
Cenário	Gerir Veículo
Caso de teste	Tentar cadastrar um veículo sem informar a sua placa
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Estar logado como motorista 1.1. Acesse a área de meus veículos 1.2. Selecionar a opção Adicionar novo Carro 1.3. Preencha as informações de (modelo e número de assentos), sem preencher o campo "Placa: ". 1.4 Clique em Cadastrar 1.5 Sistema retorna a mensagem "Placa Inválida "

Resultado esperado	O sistema deve permitir que o usuário logado como motorista não consiga cadastrar um veículo sem informar sua placa
Resultado obtido	O sistema não permitiu que o usuário logado como motorista conseguisse cadastrar um carro sem informar a placa do veículo
Status	Aprovado
Observação	Evidências: 📁 Teste ID 2 Placa.mov

ID	3.1
Cenário	Administração de caronas
Caso de teste	Consultar uma carona existente
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0. Acesse a opção "Home" como passageiro. 1.1. Selecione a opção "Explorar caronas" 1.2. Selecione uma carona existente na lista 1.4. Selecione a opção "Detalhes" 1.5. Verifique os dados exibidos (origem, destino, vagas)
Resultado esperado	O sistema deve exibir corretamente para o usuário logado como passageiro todos os dados da carona selecionada
Resultado obtido	O sistema exibiu para o usuário logado como passageiro corretamente todos os dados da carona selecionada
Status	Aprovada
Observação	Evidências: 📁 3.1CasoTesteFuncionalidade.mp4

ID	3.2
Cenário	Administração de caronas
Caso de teste	Adicionar uma nova carona no sistema
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0. Acesse a área de "Home" como motorista 1.1. Selecione a opção "Criar nova carona" 1.2. Preencha as informações (origem, destino, vagas) 1.3. Clique em "Criar carona"

Resultado esperado	O sistema deve permitir que o usuário logado como motorista consiga adicionar uma nova carona
Resultado obtido	O sistema permitiu que o usuário logado como motorista consiga adicionar uma nova carona com sucesso
Status	Aprovada
Observação	Evidências: 📺 3.2CasoTesteFuncionalidade.mp4

ID	3.3
Cenário	Administração de caronas
Caso de teste	Atualizar dados de uma carona cadastrada
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0. Acesse a área de "Home" como "motorista" 1.1. Selecione a opção "Gerenciar minhas caronas" 1.2. Selecione a opção "Editar" 1.2. Altere as informações desejadas (status, origem, destino, vagas) 1.3. Clique em "Salvar alterações"
Resultado esperado	O sistema deve permitir que o usuário logado como motorista consiga atualizar os dados de uma carona já cadastrada
Resultado obtido	O sistema permitiu que o usuário logado como motorista atualizasse os dados da carona desejada
Status	Aprovada
Observação	Evidências: 📺 3.3CasoTesteFuncionalidade.mp4

ID	3.4
Cenário	Administração de caronas
Caso de teste	Excluir uma carona cadastrada
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0. Acesse a área de "Home" 1.1. Selecione a opção "Gerenciar minhas caronas" 1.2. Selecione a opção "Excluir" na carona desejada

Resultado esperado	O sistema deve permitir que o usuário logado como motorista consiga excluir uma carona já cadastrada
Resultado obtido	O sistema permitiu que o usuário logado como motorista excluísse a carona selecionada
Status	Aprovada
Observação	Evidências: 📺 3.4CasoTesteFuncionalidade.mp4

ID	4.1
Cenário	Candidatura (adicionar, consultar, atualizar)
Caso de teste	Realizar uma candidatura e acompanhar seu status até o fim da corrida
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Acessar Explorar caronas 1.1 Selecionar Detalhes de uma carona 1.2 Selecionar Reservar carona 1.3 Escrever mensagem para o motorista 1.4 Selecionar Enviar 1.5 Selecionar home 1.6 Selecionar Minhas Aplicações
Resultado esperado	Realizar aplicação e acompanhar seu status até sua conclusão
Resultado obtido	Candidatura realizada
Status	Aprovado
Observação	Evidências: 📺 Teste4.1.mp4

ID	4.3
Cenário	Caronas Disponíveis (filtrar)
Caso de teste	Testar a funcionalidade de filtrar caronas
Prioridade	Baixa
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Abrir a API base(http://localhost/pucaronas/api/carona.php) 1.1 Selecionar o método origem

	(http://localhost/pucaronas/api/carona.php?origem=Portao) 1.2 Selecionar o método destino (http://localhost/pucaronas/api/carona.php?destino=Prado) 1.3 Selecionar ambos os métodos (http://localhost/pucaronas/api/carona.php?origem=Portao&destino=Hauer)
Resultado esperado	Métodos de filtragem retornarem apenas as caronas que atendam os parâmetros passados
Resultado obtido	Apenas caronas que continham os parâmetros passados foram retornadas
Status	Aprovado
Observação	Evidências: 📺 Teste4.3.mp4

ID	4.8
Cenário	Administração de caronas por Administrador
Caso de teste	Excluir uma carona cadastrada por qualquer usuário sendo um Administrador
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.1 Realizar Login como ADM 1.2 Deletar a carona de um usuário 1.3 Verificar com o Usuário que teve a carona deletada
Resultado esperado	O sistema deve permitir que o usuário logado como Administrador consiga excluir qualquer carona cadastrada no sistema.
Resultado obtido	O sistema permite que o ADM exclua a carona.
Status	Aprovado
Observação	Evidências: 📺 Teste4.8.mp4

ID	5.1
Cenário	Avaliação de corrida
Caso de teste	Adicionar e consultar uma avaliação de corrida
Prioridade	Alta

Tipo de teste	Funcionalidade
Passo-a-passo para execução	1. Acesse a página de corridas 2. Selecione uma corrida disponível 3. Adicione uma avaliação para a corrida 4. Informe a nota e/ou comentário solicitado 5. Salve a avaliação 6. Acesse novamente os dados da corrida 7. Consulte a avaliação adicionada
Resultado esperado	O sistema deve permitir adicionar a avaliação e, posteriormente, exibir a avaliação cadastrada ao consultar os dados da corrida.
Resultado obtido	A avaliação foi adicionada e exibida corretamente na consulta da corrida.
Status	Aprovada
Observação	Evidências: 📹 5.1CasoTesteFuncionalidade.mp4

ID	5.3
Cenário	Cadastro de avaliações
Caso de teste	Avaliar uma corrida informando nota acima do permitido
Prioridade	Média
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1. Fazer login e acessar a área de corridas 2. Selecionar uma corrida finalizada que ainda não tenha sido avaliada 3. Selecionar a opção de avaliar 4. Informar a nota 10 (acima do máximo permitido, que é 5) 5. Tentar salvar a avaliação
Resultado esperado	O sistema não deve permitir a avaliação e deve exibir a mensagem "Nota inválida!"
Resultado obtido	O sistema exibiu "Nota inválida!!" e não permitiu adicionar a avaliação acima de 5
Status	Aprovado
Observação	Evidências: 📹 5.3CasoTesteFuncionalidade.mp4(00:00-00:15)

ID	5.7
----	-----

Cenário	Avaliação de corrida
Caso de teste	Tentar adicionar uma avaliação sem preencher os campos obrigatórios
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1. Faça login no sistema 2. Acesse uma corrida disponível 3. Acesse a opção de adicionar avaliação 4. Deixe os campos obrigatórios sem preenchimento 5. Tente salvar a avaliação 6. Verifique a resposta apresentada pelo sistema
Resultado esperado	O sistema não deve cadastrar a avaliação e deve informar que os campos obrigatórios precisam ser preenchidos.
Resultado obtido	O sistema impediu o cadastro da avaliação com campos obrigatórios vazios.
Status	Aprovado
Observação	Evidências: 📹 Teste5.7.mp4

ID	6.1
Cenário	Consulta de corridas pendentes como motorista
Caso de teste	Consultar corrida(s) como motorista
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Fazer o login como motorista 1.2 Consultar a tela de corridas
Resultado esperado	Sistema mostra as corridas do motorista
Resultado obtido	O software mostrou a corrida para o motorista
Status	Aprovado
Observação	Evidências: 📹 6.1CasoTesteFuncionalidade.mp4

ID	6.2
Cenário	Consulta de corridas pendentes como passageiro

Caso de teste	Consultar corrida(s) como passageiro
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Fazer o login como passageiro 1.2 Consultar a tela de corridas
Resultado esperado	Sistema mostra as corridas para o passageiro
Resultado obtido	O software mostrou a corrida para o passageiro
Status	Aprovado
Observação	Evidências: 📹 6.2CasoTesteFuncionalidade.mp4

ID	6.3
Cenário	Atualização de dados da corrida pendente
Caso de teste	Atualizar informações de uma corrida existente (ex: origem, destino)
Prioridade	Média
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Selecionar uma corrida existente 1.1 Alterar um ou mais campos válidos 1.2 Confirmar a atualização
Resultado esperado	Dados da corrida são atualizados corretamente
Resultado obtido	É possível mudar o nome da corrida porém origem e destino não alteram na página de corridas
Status	Reprovado
Observação	Evidências: 📹 6.3CasoTesteFuncionalidade.mp4

ID	6.4
Cenário	Exclusão de corridas pendentes
Caso de teste	Excluir uma corrida pendente
Prioridade	Alta
Tipo de teste	Funcionalidade
Passo-a-passo para execução	1.0 Selecionar uma corrida existente 1.1 Executar a ação de exclusão 1.2 Tentar consultar a corrida excluída
Resultado esperado	Corrida é removida da página
Resultado obtido	A corrida foi removida da página, mas a carona correspondente também foi excluída.
Status	Reprovado
Observação	Evidências: 📹 6.4CasoTesteFuncionalidade.mp4 📹 6.4(2)CasoTesteFuncionalidade.mp4 (Evidência no banco)

4.2 Casos de portabilidade

ID	4.7
Cenário	Acesso e navegação no sistema via dispositivos móveis.
Caso de teste	Validar se a interface gráfica, formulários e elementos interativos se adaptam responsivamente a telas de dispositivos moveis.
Prioridade	Alta
Tipo de teste	Portabilidade
Passo-a-passo para execução	1.1 Acesse o sistema no Google Chrome 1.2 Abra o DevTools 1.3 Ative a Device Toolbar 1.4 Cadastre-se no site 1.5 Navegue pelas telas principais: Explorar Caronas, Detalhes da Carona, Reservar Carona e Meus Veículos. 1.6 Tente interagir com os botões de ação
Resultado esperado	O conteúdo e os cards devem se reorganizar verticalmente de forma fluida (<i>layout flexível/grid</i>). Todos os botões, links e campos interativos devem manter dimensões mínimas adequadas para toque (pelo menos 48x48px recomendados para áreas de toque).

	A troca de orientação (Retrato/Paisagem) não deve sobrepor nem cortar elementos de formulário ou botões de confirmação.
Resultado obtido	Foi possível navegar pelo site porém com usabilidade comprometida pois em algumas telas a navbar não responde corretamente.
Status	Aprovado com Ressalvas
Observação	Evidências:  teste4.7.mp4 A barra de navegação (<i>navbar</i>) não disponibiliza um atalho direto para retornar à página inicial (<i>Home</i>) ou ao painel principal em determinadas seções da aplicação.

4.3 Casos de desempenho

ID	2.5
Cenário	Listar Carros cadastrados no sistema
Caso de teste	Realizar o teste de desempenho da página responsável por listar todos os carros que foram cadastrados pelo usuário motorista
Prioridade	Média
Tipo de teste	Desempenho
Passo-a-passo para execução	1.0 Estar logado como usuário motorista 1.1 Selecionar a página HOME 1.2 Selecionar a opção MEUS VEÍCULOS 1.3 Adicionar veículos para realização do teste 1.4 Rodar o teste utilizando a ferramenta do Google Lighthouse com a função de teste de desempenho ativada
Resultado esperado	First Contentful Paint (FCP) < 1,8 s Largest Contentful Paint (LCP) < 2,5 s Total Blocking Time (TBT) < 300 ms Cumulative Layout Shift (CLS) < 0,10 Speed Index (SI) < 3,4 s
Resultado obtido	First Contentful Paint (FCP) = 0,5 s Largest Contentful Paint (LCP) = 0,5 s Total Blocking Time (TBT) = 0 ms Cumulative Layout Shift (CLS) = 0 Speed Index (SI) = 0,5
Status	Aprovado
Observação	Evidências:  testeDesempenho2.5.pdf

ID	2.6
----	-----

Cenário	Editar veículo na página Meus Veículos
Caso de teste	Realizar o teste de desempenho da página responsável por editar os dados de um veículo cadastrado no sistema.
Prioridade	Média
Tipo de teste	Desempenho
Passo-a-passo para execução	1.0 Estar logado como usuário motorista 1.1 Selecionar a página HOME 1.2 Selecionar a opção MEUS VEÍCULOS 1.3 Adicionar veículos para realização do teste 1.4 Em um dos veículos adicionados clicar em EDITAR 1.5 Rodar o teste utilizando a ferramenta do Google Lighthouse com a função de teste de desempenho ativada
Resultado esperado	First Contentful Paint (FCP) < 1,8 s Largest Contentful Paint (LCP) < 2,5 s Total Blocking Time (TBT) < 300 ms Cumulative Layout Shift (CLS) < 0,10 Speed Index (SI) < 3,4 s
Resultado obtido	First Contentful Paint (FCP) = 0,5 s Largest Contentful Paint (LCP) = 0,5 s Total Blocking Time (TBT) = 0 ms Cumulative Layout Shift (CLS) = 0 Speed Index (SI) = 0,5
Status	Aprovado
Observação	Evidências:  TesteDesempenho2.6.pdf

ID	3.5
Cenário	Administração de Caronas
Caso de teste	Verificar o tempo de carregamento da listagem de caronas
Prioridade	Alta
Tipo de teste	Desempenho
Passo-a-passo para execução	1.0 Realizar o login no sistema como passageiro. 1.1 Acessar a tela de Caronas Disponíveis. 1.2 Utilizar o DevTools e o Lighthouse para analisar o desempenho da página durante o carregamento da listagem de caronas. 1.3 Verificar o tempo necessário para que a listagem de caronas seja carregada e disponibilizada para interação.

Resultado esperado	A lista deve carregar em até 2 segundos
Resultado obtido	A listagem de caronas apresentou tempo de carregamento de aproximadamente 0,8 segundo, conforme análise realizada pelo DevTools e o Lighthouse.
Status	Aprovado
Observação	Evidências: 📹 3.5CasoTesteDesempenho.mp4

ID	3.6
Cenário	Administração de Caronas
Caso de teste	Desempenho ao consultar à página de administração de caronas
Prioridade	Alta
Tipo de teste	Desempenho
Passo-a-passo para execução	1.0 Realizar o login no sistema como administrador. 1.1 Acessar a página “Mostrar todas as caronas” disponível no menu do sistema. 1.2 Utilizar o DevTools e o Lighthouse para analisar o desempenho da página durante o carregamento da lista das caronas. 1.3 Verificar o tempo necessário para que a lista das caronas seja carregada e disponibilizada para interação.
Resultado esperado	A página de administração de caronas deve ser carregada e disponibilizada em até 2 segundos.
Resultado obtido	A página de administração de caronas apresentou tempo de carregamento de aproximadamente 0,8 segundo, conforme análise realizada pelo DevTools e pelo Lighthouse.
Status	Aprovado
Observação	Evidências: 📹 3.6CasoTesteDesempenho.mp4

ID	4.2
Cenário	Testar o desempenho da página minhas aplicações
Caso de teste	Candidaturas
Prioridade	Média
Tipo de teste	Desempenho

Passo-a-passo para execução	1.0 Selecionar Minhas Aplicações 1.1 Abrir o DevTools 1.2 Selecionar o Google lighthouse 1.3 Gerar um teste de performance para desktop
Resultado esperado	Site receber nota igual ou superior a 90 de pontuação
Resultado obtido	Site atingiu 100 contendo sugestões de melhoria não obrigatórias
Status	Aprovado
Observação	Evidências: 📹 Teste4.2.mp4

ID	5.2
Cenário	Listagem de corridas
Caso de teste	Verificar o desempenho da listagem de corridas
Prioridade	Alta
Tipo de teste	Desempenho
Passo-a-passo para execução	1. Acesse a página de corridas 2. Solicite a listagem das corridas disponíveis 3. Aguarde o carregamento dos resultados 4. Verifique o tempo necessário para que a lista seja exibida 5. Repita a operação para verificar a consistência do tempo de resposta
Resultado esperado	O sistema deve listar as corridas corretamente, apresentando os resultados dentro de um tempo de resposta adequado, sem travamentos ou erros.
Resultado obtido	O sistema listou as corridas corretamente, com um tempo de respostas consistente inferior a 2 segundos. Evidência: 📹 5.2TesteFuncionalidade.mp4
Status	Aprovado
Observação	N/A

4.4 Casos de Unidade

ID	1.5
Cenário	Login de usuário

Caso de teste	Requisição de API com senha incorreta
Prioridade	Alta
Tipo de teste	API - Unidade
Passo-a-passo para execução	1.0 Criar requisição de API no método POST 1.1 Usar caminho <code>http://localhost/pucaronas/api/usuario.php?action=login</code> 1.2 Inserir como param { "email": "ADM@pucpr.edu.br", "senha_hash": "teste" } 1.3 Clicar em enviar requisição
Resultado esperado	Deve retornar um JSON encode com a mensagem: "Email ou senha inválidos"
Resultado obtido	Retornou json encode: { "error": "Email ou senha inválidos" }
Status	Aprovado
Observação	Evidências:  1.5.png

ID	1.6
Cenário	Login de usuário
Caso de teste	Requisição de API com senha correta
Prioridade	Alta
Tipo de teste	API - Unidade
Passo-a-passo para execução	1.0 Criar requisição de API no método POST 1.1 Usar caminho <code>http://localhost/pucaronas/api/usuario.php?action=login</code> 1.2 Inserir como param{ "email": "ADM@pucpr.edu.br", "senha_hash": "ADM12345" } 1.3 Clicar em enviar requisição
Resultado esperado	Deve retornar um JSON encode com a mensagem: "adm login ok"
Resultado obtido	Retornou um json com mensagem e nome

Status	Aprovado
Observação	Evidências:  1.6.png

ID	1.7
Cenário	Cadastro de usuário
Caso de teste	Requisição de API sem body
Prioridade	Média
Tipo de teste	API - Unidade
Passo-a-passo para execução	1.0 Criar requisição de API no método POST 1.1 Usar caminho <code>http://localhost/pucaronas/api/usuario.php?action=login</code> 1.2 Deixar o body limpo. 1.3 Clicar em enviar requisição
Resultado esperado	Deve retornar apenas json com o erro da conexão SQL como definido em código
Resultado obtido	Retornou um json com "error": "E-mail já cadastrado." e também retornou múltiplos PHP notices.
Status	Reprovado
Observação	Evidências:  1.7.png

4.5 Casos unitários

ID	2.7
Cenário	Procedure aprovar_usuario
Caso de teste	Testar o funcionamento da Procedure aprovar_usuario
Prioridade	Alta
Tipo de teste	Unitário
Passo-a-passo para execução	1.0 Criar um usuário de teste com status espera 1.1 Executar CALL aprovar_usuario(id) 1.2 Consultar o status do usuário. 1.3 Comparar o resultado com o esperado usando um IF/ELSE (Assert).

Resultado esperado	O status do usuário deve ser alterado de espera para aprovado, exibindo "Passou no teste..."
Resultado obtido	O usuário teve seu status alterado para aprovado e o teste exibiu "Passou no teste..."
Status	Aprovado
Observação	Evidências: 📁 Teste2_7.mov

ID	3.8
Cenário	Administração de Caronas
Caso de teste	Procedure deletar_carona
Prioridade	Alta
Tipo de teste	Unitário
Passo-a-passo para execução	1.0 Ver a tabela carona (aplicacao/corrida se tiver ligação) antes de excluir 1.1 Excluir a carona pelo site 1.2 Ver de novo a tabela depois para comparar
Resultado esperado	Carona some da tabela e não deixa nada quebrado nas tabelas ligadas
Resultado obtido	Carona foi removida e as tabelas relacionadas ficaram consistentes
Status	Aprovado
Observação	Evidências: 📁 3.8CasoTesteUnitario.mp4

ID	4.4
Cenário	Trigger aplicacao_corrida
Caso de teste	Testar o funcionamento da trigger aplicacao_corrida
Prioridade	Alta
Tipo de teste	Unitário
Passo-a-passo para execução	1.0 Gerar o cenário necessário para realizar o teste 1.1 Executar o teste 1.2 Verificar o Resultado
Resultado esperado	Gerar a corrida e quantidade de vagas decrementadas

Resultado obtido	Corrida foi gerada e vagas foram decrementadas
Status	Aprovado
Observação	Evidências: 📹 Teste4.4.mp4

ID	4.5
Cenário	Cadastro de carona por usuário inválido.
Caso de teste	Tentar cadastrar uma carona diretamente no banco associando um usuário com status 'banido'
Prioridade	Média
Tipo de teste	Unitário
Passo-a-passo para execução	1.0 Criar um usuário motorista banido 1.1 Criar uma carona com este usuário 1.2 Verificar o retorno no banco
Resultado esperado	A transação do banco deve disparar um erro impedindo o cadastro da carona por usuários não aprovados.
Resultado obtido	A carona foi cadastrada no banco de dados com sucesso para o motorista com o status 'banido'
Status	Reprovado
Observação	Evidências: 📹 Teste4.5.mp4 Verificação não existe no banco de dados

ID	4.6
Cenário	Exclusão de usuário no banco de dados.
Caso de teste	Verificar a exclusão em cascata ao executar a procedure deletar_usuario para motorista com histórico no sistema.
Prioridade	Alta
Tipo de teste	Unitário
Passo-a-passo para execução	1.0 Criar um usuário no banco que possua veículos cadastrados, caronas criadas, candidaturas e avaliações. 1.1 Verificar as tabelas usuário, veículo, carona, candidatura e avaliação para confirmar a presença dos registros associados ao id do usuário. 1.2 Executar o comando SQL:

	CALL deletar_usuario(<id_do_usuario>); 1.3 Consultar novamente todas as tabelas citadas filtrando pelo id do usuário deletado.
Resultado esperado	O usuário deve ser removido com sucesso sem violar chaves estrangeiras. Os registros dependentes devem ser removidos em cascata conforme a regra de integridade do modelo relacional.
Resultado obtido	Ao executar a procedure apresentou erro gerado pela procedure não inclui a instrução de remoção da tabela avaliação
Status	Reprovado
Observação	Evidências: 📹 Teste4.5.mp4 Necessário revisar a procedure para deletar a avaliação

ID	5.4
Cenário	Atualização do status do usuário
Caso de teste	Verificar o funcionamento do trigger usuario_status durante a alteração do status de um usuário
Prioridade	Alta
Tipo de teste	Unitário
Passo-a-passo para execução	1. No banco, criar um usuário com status 'espera' 2. Validar se o log antes está vazio: SELECT * FROM log_usuario_status WHERE id_usuario = <id>; (deve estar vazio) 3. Executar UPDATE usuario SET status = 'aprovado' WHERE id = <id>; 4. Consultar de volta: SELECT * FROM log_usuario_status WHERE id_usuario = <id>;
Resultado esperado	Deve existir uma nova linha registrando a mudança de 'espera' para 'aprovado'
Resultado obtido	Não houve atualização no log de usuário, a procedure deve ser ajustada
Status	Reprovado
Observação	Evidências: 1.

*localhost> Script X

```
use pucaronas;
INSERT INTO usuario (nome, email, senha_hash, data_nasc, status, cargo, doc)
VALUES ('Teste Trigger 54', 'teste54@pucpr.edu.br',
'$2y$10$abcdefghijklmnopqrstuv', '2000-01-01', 'espera', 'passageiro', NULL);
```

Statistics 1 X

Name	Value
Queries	2
Updated Rows	1
Execute time	0.01s
Fetch time	0s
Total time	0.01s
Start time	2026-10-01 16:31:04.337
Finish time	2026-10-01 16:31:04.350

2.

```
use pucaronas;
SELECT id FROM usuario WHERE email = 'teste54@pucpr.edu.br';
```

usuario 1 X Statistics 1

id
3

3.

*localhost> Script X

```
use pucaronas;
SELECT * FROM log_usuario_status WHERE id_usuario = 3;
```

log_usuario_status 1 X Statistics 1

id_usuario	status_anterior	status_novo
3		

```
use pucaronas;
UPDATE usuario SET status = 'aprovado' WHERE id = 3;
```

Statistics 1 X

Name	Value
Queries	2
Updated Rows	1
Execute time	0.011s
Fetch time	0s
Total time	0.011s
Start time	2026-10-01 16:36:57.953
Finish time	2026-10-01 16:36:57.967

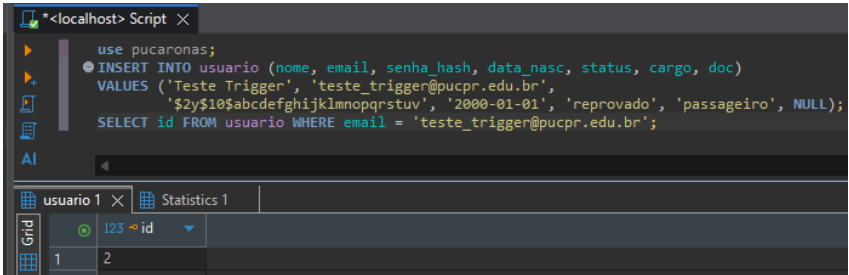
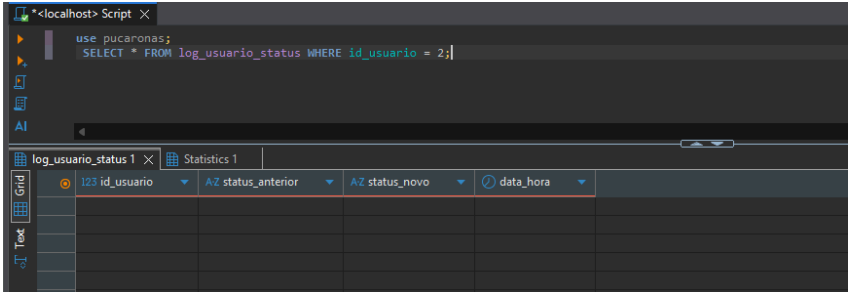
4.

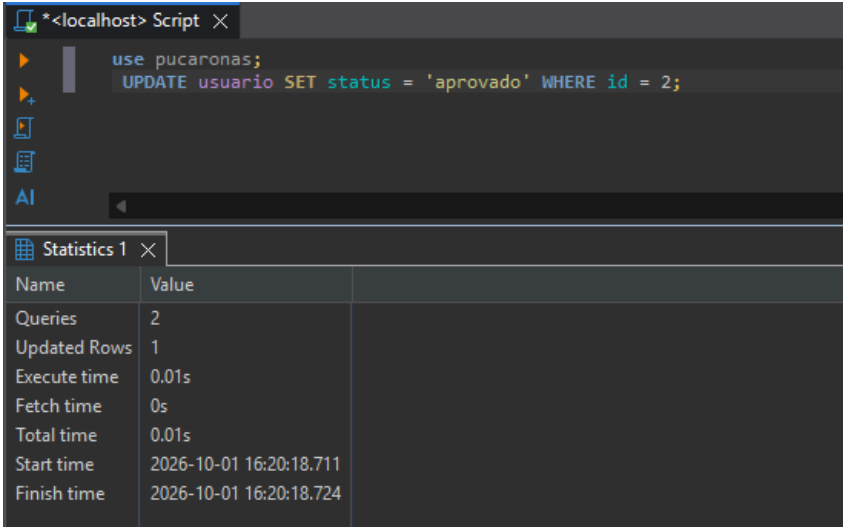
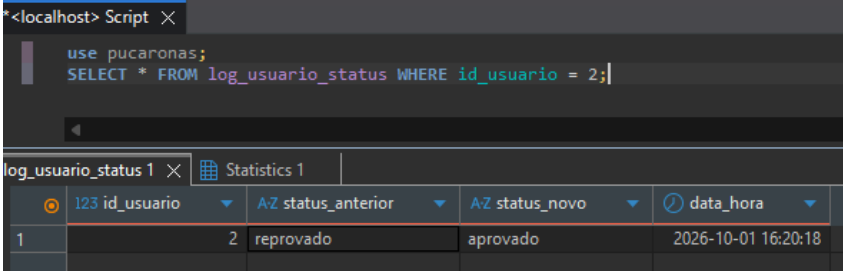
*localhost> Script X

```
use pucaronas;
SELECT * FROM log_usuario_status WHERE id_usuario = 3;
```

log_usuario_status 1 X Statistics 1

id_usuario	status_anterior	status_novo	data_hora
3			

ID	5.5
Cenário	Alteração sucessiva do status do usuário
Caso de teste	Verificar o comportamento da trigger na aprovação de um usuário vindo de outro status
Prioridade	Alta
Tipo de teste	Unitário
Passo-a-passo para execução	1. No banco, criar um usuário e levá-lo ao status 'reprovado' (ou 'banido') 2. Conferir o log antes: <code>SELECT * FROM log_usuario_status WHERE id_usuario = <id>;</code> 3. Executar <code>UPDATE usuario SET status = 'aprovado' WHERE id = <id>;</code> 4. Consultar de novo: <code>SELECT * FROM log_usuario_status WHERE id_usuario = <id>;</code>
Resultado esperado	Deve existir uma nova linha no log registrando a mudança para 'aprovado'.
Resultado obtido	O status foi atualizado corretamente em todas as alterações realizadas. Evidência: 1.  The screenshot shows a SQL script being executed in a tool. The script includes: <code>use pucaronas;</code>, <code>INSERT INTO usuario (nome, email, senha_hash, data_nasc, status, cargo, doc) VALUES ('Teste Trigger', 'teste_trigger@pucpr.edu.br', '\$2y\$10\$abcdefghijklmnopqrstuv', '2000-01-01', 'reprovado', 'passageiro', NULL);</code>, and <code>SELECT id FROM usuario WHERE email = 'teste_trigger@pucpr.edu.br';</code>. Below the script, a table view for 'usuario 1' is shown with columns 'id' and 'status'. The first row has 'id' 1 and 'status' 2. 2.  The screenshot shows a SQL query being executed: <code>SELECT * FROM log_usuario_status WHERE id_usuario = 2;</code>. Below the query, a table view for 'log_usuario_status 1' is shown with columns 'id_usuario', 'status_anterior', 'status_novo', and 'data_hora'. The first row has 'id_usuario' 123, 'status_anterior' A2, 'status_novo' A2, and 'data_hora'. 3.

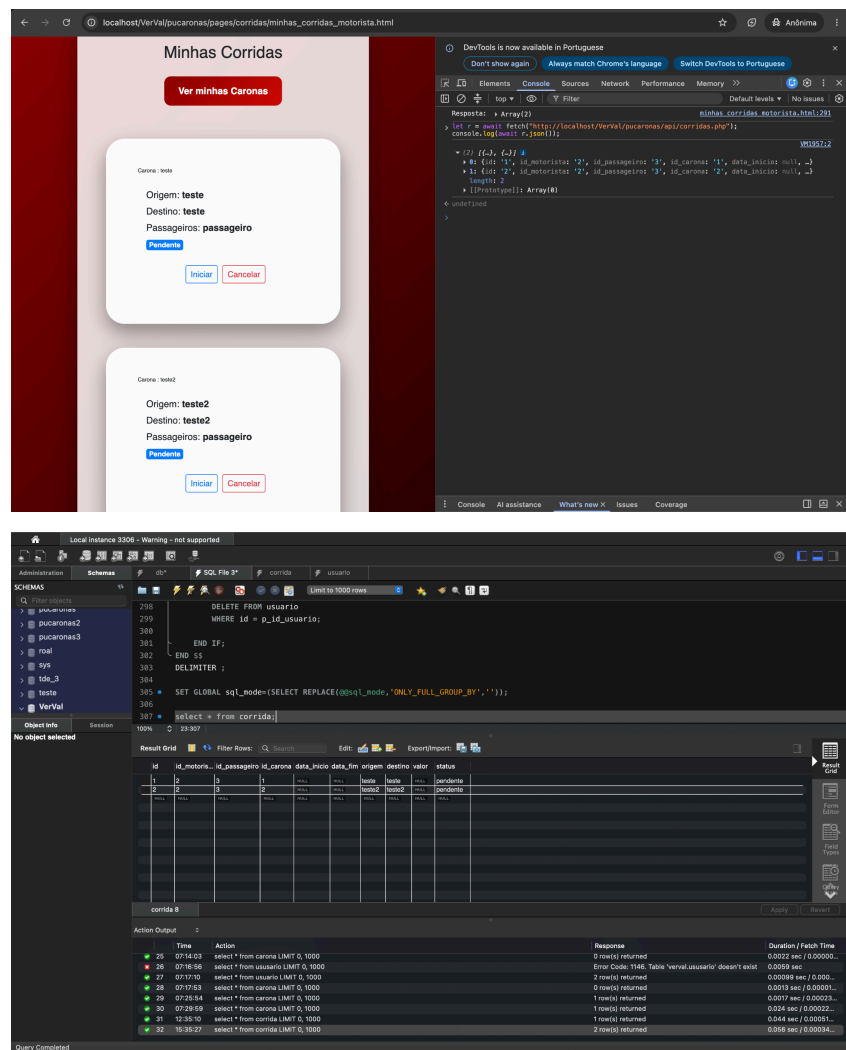
	 4. 
Status	Aprovado
Observação	

ID	5.8
Cenário	Avaliação de corrida
Caso de teste	Enviar avaliação com nota acima do permitido via API
Prioridade	Alta
Tipo de teste	Api - Unidade
Passo-a-passo para execução	1.1 Abrir o DevTools (F12) no site pucaronas e selecionar a aba Console 1.2 Colar o código abaixo e apertar Enter: <pre>let r = await fetch("http://localhost/pucaronas/api/avaliacoes.php?id_passageiro=1&id_motorista=2", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({</pre>

	nota: 99, tipo: "passageiro_para_motorista", comentario: "teste nota invalida" }) }); console.log(await r.json());
Resultado esperado	O sistema deve rejeitar a nota fora do intervalo permitido (1 a 5), informando erro, e não registrar a avaliação.
Resultado obtido	O sistema retornou um erro. <pre> >> let r = await fetch("http://localhost/pucaronas/api/avaliacoes.php?id_passageiro=1&id_motorista=2", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ nota: 99, }) }); Object { error: "CONSTRAINT `avaliacao.nota` failed for `pucaronas`.`avaliacao`" } debugger eval code:14:18 < undefined </pre>
Status	Aprovado
Observação	A coluna nota tem CHECK (nota BETWEEN 1 AND 5), mas a API não valida antes do INSERT.

ID	6.5
Cenário	Administração de corridas
Caso de teste	Requisição de API para consultar corridas via console
Prioridade	Alta
Tipo de teste	API
Passo-a-passo para execução	1.0 Fazer login e acessar o site (localhost) 1.1 Abrir o DevTools (F12) e selecionar a aba Console 1.2 Colar o código abaixo e apertar Enter: let r = await fetch("http://localhost/VerVal/pucaronas/api/corridas.php"); console.log(await r.json());
Resultado esperado	O console deve exibir um JSON com a lista de corridas cadastradas.
Resultado obtido	Foi exibido o JSON com a lista de corridas cadastradas. <pre> >> let r = await fetch("http://localhost/pucaronas/api/corridas.php"); console.log(await r.json()); Array(500) [{}, {}, {}, {}, {}, {}, {}, {}, {}, {}, ...] debugger eval code:4:18 [0_99] 0: Object { id: "1", id_motorista: "201", id_passageiro: "202", ... } 1: Object { id: "2", id_motorista: "201", id_passageiro: "202", ... } 2: Object { id: "3", id_motorista: "201", id_passageiro: "202", ... } </pre>
Status	Aprovado

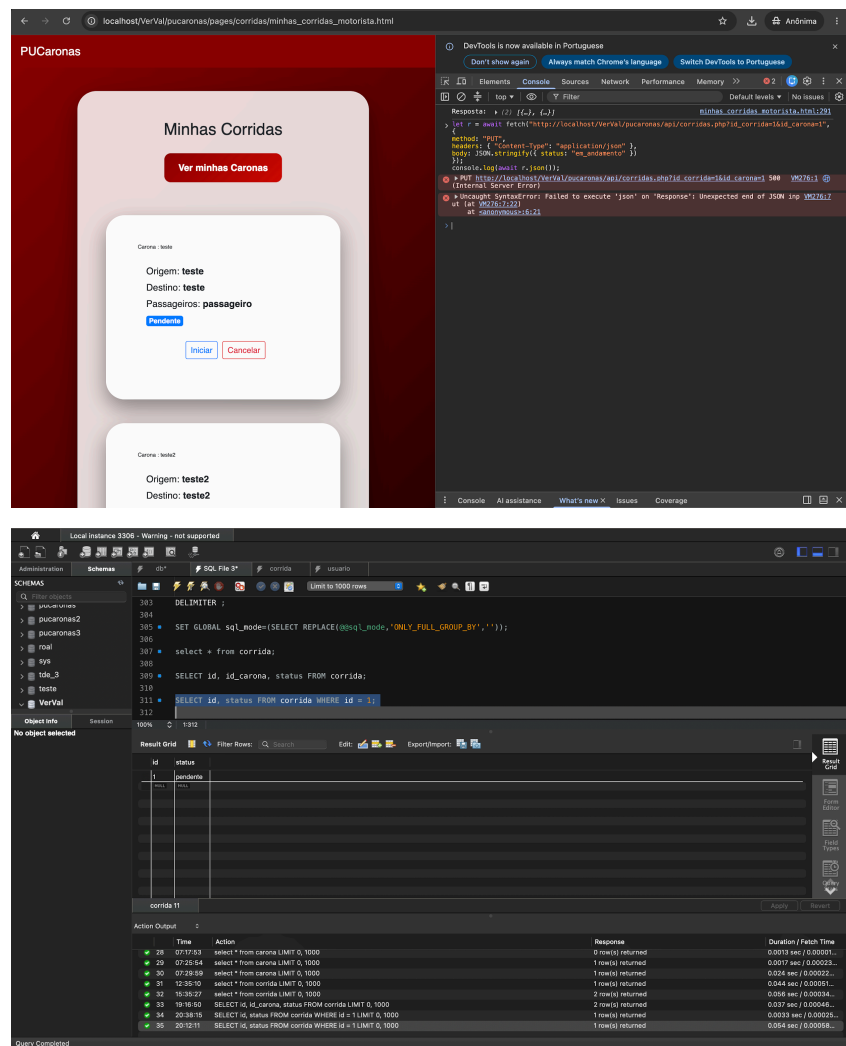
Observação



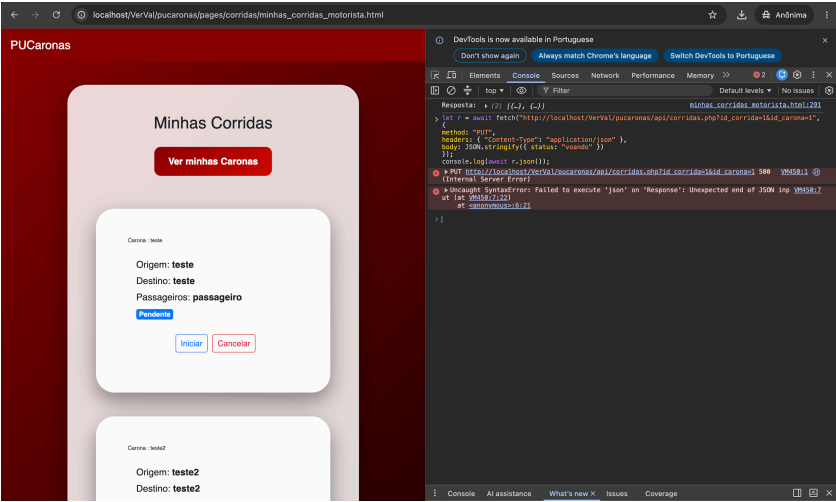
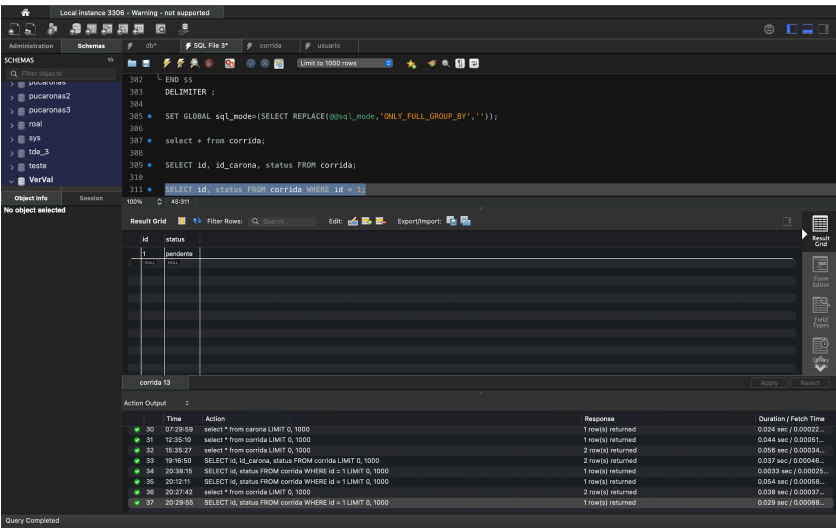
ID	6.6
Cenário	Administração de corridas
Caso de teste	Requisição de API para atualizar o status de uma corrida via console
Prioridade	Alta
Tipo de teste	API

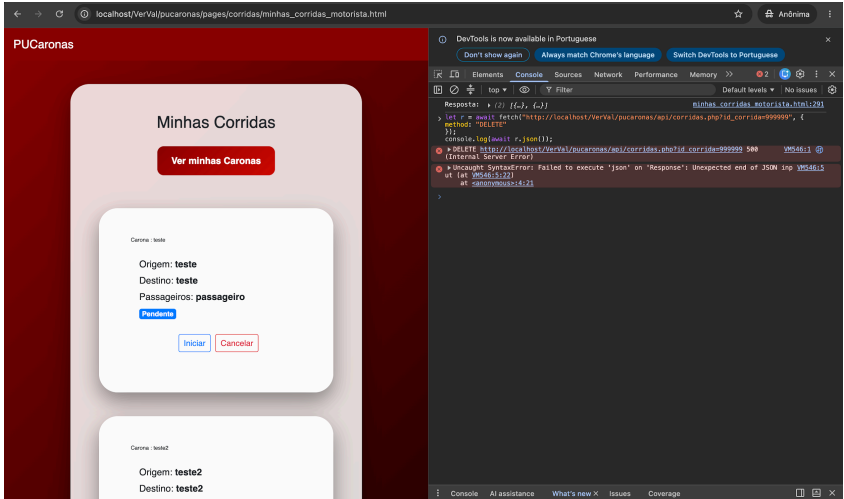
Passo-a-passo para execução	1.0 Fazer login e acessar o site (localhost) 1.1 Abrir o DevTools (F12) e selecionar a aba Console 1.2 Colar o código abaixo (corrida de id 1, conferida com SELECT id, id_carona, status FROM corrida;) e apertar Enter: <pre>let r = await fetch("http://localhost/VerVal/pucaronas/api/corridas.php?id_corri da=1&id_carona=1", { method: "PUT", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ status: "em_andamento" }) }); console.log(await r.json());</pre> 1.3 Consultar no banco: SELECT id, status FROM corrida WHERE id = 1;
Resultado esperado	A resposta deve indicar sucesso e o status da corrida deve estar alterado no banco.
Resultado obtido	A API retornou erro 500 (Internal Server Error) e não atualizou o status. O banco continua pendente .
Status	Reprovado

Observação



ID	6.7
Cenário	Administração de corridas
Caso de teste	Requisição de API para atualizar corrida com status inválido via console
Prioridade	Alta
Tipo de teste	API

Passo-a-passo para execução	1.0 Fazer login e acessar o site (localhost) 1.1 Abrir o DevTools (F12) e selecionar a aba Console 1.2 Colar o código abaixo e apertar Enter: <pre>let r = await fetch("http://localhost/VerVal/pucaronas/api/corridas.php?id_corrida=1&id_carona=1", { method: "PUT", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ status: "voando" }) }); console.log(await r.json());</pre>
Resultado esperado	O sistema deve rejeitar o status inválido, informando erro, e não deve alterar a corrida.
Resultado obtido	A API retornou erro 500 em vez de uma mensagem de status inválido. O banco continua pendente .
Status	Reprovado
Observação	 

ID	6.8
Cenário	Administração de corridas
Caso de teste	Requisição de API para excluir uma corrida inexistente via console
Prioridade	Média
Tipo de teste	API
Passo-a-passo para execução	1.0 Fazer login e acessar o site (localhost) 1.1 Abrir o DevTools (F12) e selecionar a aba Console 1.2 Colar o código abaixo e apertar Enter: <pre>let r = await fetch("http://localhost/VerVal/pucaronas/api/corridas.php?id_corri da=9999999", { method: "DELETE" }); console.log(await r.json());</pre>
Resultado esperado	O sistema deve informar que a corrida não existe, sem erro interno.
Resultado obtido	A API retornou erro 500 em vez de informar que a corrida não existe.
Status	Reprovado
Observação	

4.6 Casos de Acessibilidade

ID	2.8
----	-----

Cenário	Acessibilidade da página Meus Carros
Caso de teste	Testar e avaliar o quesito acessibilidade na página responsável por listar os carros cadastrados pelo usuário motorista
Prioridade	Baixa
Tipo de teste	Acessibilidade
Passo-a-passo para execução	1.0 Estar logado no sistema como usuário motorista 1.1 Acessar a página Meus Veículos 1.2 Abrir o Lighthouse no DevTools. 1.3. Selecionar Accessibility. 1.4. Executar a análise. 1.5. Registrar os problemas de acessibilidade encontrados.
Resultado esperado	Nota geral de Acessibilidade da página acima de 70 pontos
Resultado obtido	Nota obtida: 73
Status	Aprovado
Observação	Evidências:  Teste2.8.pdf

ID	3.7
Cenário	Acessibilidade da página de Administração de Caronas
Caso de teste	Verificar a acessibilidade da página de consulta de caronas como administrador
Prioridade	Média
Tipo de teste	Acessibilidade
Passo-a-passo para execução	1.0 Acessar o sistema logado como administrador 1.1 Acessar a página de Administração de Caronas. 1.2 Abrir o DevTools do navegador. 1.3 Acessar a ferramenta Lighthouse e selecionar a categoria Acessibilidade. 1.4 Executar a análise da página. 1.5 Registrar a pontuação obtida e os possíveis problemas de acessibilidade identificados pelo Lighthouse.
Resultado esperado	Nota geral de Acessibilidade da página acima de 70 pontos
Resultado obtido	Nota obtida: 90
Status	Aprovado
Observação	Evidência:  3.7CasoTesteAcessibilidade.pdf

4.7 Casos de Segurança

ID	1.8
Cenário	Criptografia de Senha de usuários
Caso de teste	Acessar automacao.php e criptografar todos os campos, corrompendo todas senhas.
Prioridade	Alta
Tipo de teste	Segurança
Passo-a-passo para execução	1.0 Realizar o login no usuário administrador(email : ADM@pucpr.edu.br e senha: ADM12345 2.0 Realizar Log out 3.0 Acessar http://localhost/pucaronas/api/automacao.php no navegador 4.0 Realizar o login no usuário administrador(email : ADM@pucpr.edu.br e senha: ADM12345
Resultado esperado	Deve retornar o usuário a tela home.html e não deve criptografar as senhas do banco de dados
Resultado obtido	Não retornou a página home, não foi possível realizar o login ao usuário Admin, logo provando a criptografia da criptografia do banco de dados.
Status	Reprovado
Observação	Evidências: 🚩 1.8.mp4

4.8 Casos de Stress

ID	5.6
Cenário	Listagem de 500 corridas
Caso de teste	Verificar o desempenho da listagem com 500 corridas cadastradas
Prioridade	Alta

Tipo de teste	Stress
Passo-a-passo para execução	1. Popular a tabela corrida com 500 registros 2. Fazer login no sistema 3. Acessar a página de listagem de corridas 4. Medir o tempo de carregamento (DevTools, aba Network ou Lighthouse) 5. Verificar se todas as corridas foram exibidas corretamente
Resultado esperado	First Contentful Paint (FCP) < 1,8 s Largest Contentful Paint (LCP) < 2,5 s Total Blocking Time (TBT) < 300 ms Cumulative Layout Shift (CLS) < 0,10 Speed Index (SI) < 3,4 s
Resultado obtido	First Contentful Paint (FCP) = 2,0 s Largest Contentful Paint (LCP) = 2,2 s Total Blocking Time (TBT) = 8.940 ms Cumulative Layout Shift (CLS) = 0 Speed Index (SI) = 9,8 s
Status	Reprovado https://drive.google.com/file/d/105jWmlz3ZJSjXueh67fWgsNrBTIKSBZD/view?usp=sharing
Observação	

4. Resultado

4.1. Resultados Finais:

Foram executados 48 casos de teste cobrindo funcionalidade, desempenho, acessibilidade, portabilidade, segurança, testes unitários e testes de API, abrangendo os módulos de usuário, veículo, carona, candidatura, corrida e avaliação, além das triggers e procedures definidas no banco.

Considerando os critérios de avaliação definidos neste documento o site PUC Caronas é considerado reprovado pois houve mais de um caso de teste reprovado de prioridade alta.